Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №3
Варіант 9
На тему
“Нездатні до змін структури даних (immutability) у Python”

Виконав:
Студент групи ФЕП-22с
Обиход І. В.
Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- зрозуміти та застосувати tuple, frozenset, dataclasses(frozen=True), MappingProxyType;
- замінити мутації на повернення нових копій (pure updates);
- виміряти ефект після переходу на незмінні структури.

Теоретичні відомості:

- Незмінність дає референтну прозорість, просту композицію, безпечне читання з кількох потоків і можливість кешування.
- tuple замінює list для впорядкованих колекцій, frozenset замінює set і є хешованим.
- @dataclass(frozen=True, slots=True) забороняє змінювати поля, а оновлення робиться через dataclasses.replace.
- MappingProxyType - read-only представлення словника. Перед обгортанням джерело треба скопіювати.

Завдання (варіант 9 - Система доставки):

Незмінна модель Parcel, RoutePoint, мітки labels і метадані meta. Pure-оновлення: додавання точки маршруту, мітки, зміна статусу. Маршрут - tuple, labels - frozenset, meta - read-only.

Хід роботи

У model.py описано дві доменні сутності через @dataclass(frozen=True, slots=True): RoutePoint (місто, хаб, години) та Parcel (id, відправник, вага, статус, маршрут, мітки, метадані). Маршрут зберігається як tuple, мітки як frozenset, статус обмежено Literal, а meta створюється функцією make_meta як MappingProxyType над копією словника.

У core.py реалізовано pure-оновлення: add_route_point (конкатенація tuple), add_label і remove_label (операції frozenset), change_status з перевіркою дозволених переходів, update_meta (нова read-only мапа). Кожна функція повертає нову посилку і не змінює стару.

Як просунуті завдання реалізовано утиліту freeze для глибокого заморожування вкладених структур, кешування вартості доставки через functools.lru_cache і індекс посилок за ключем (id, labels).

Parcel і RoutePoint хешуються. MappingProxyType не хешується, тому поле meta виключено з хешу через field(hash=False), що перевірено окремим тестом.

Для порівняння часу в tests/test_performance.py написано мутабельну версію «до» (list.append, set.add) та незмінну «після» з однаковою кількістю операцій. Час виміряно через pytest --durations.

Файл model.py

model.py рядки 1-54

```python
"""Незмінна модель системи доставки (ЛР3, варіант 9)."""

from collections.abc import Mapping
from dataclasses import dataclass, field, replace
from types import MappingProxyType
from typing import Literal

Status = Literal["created", "in_transit", "delivered", "returned"]


def make_meta(**kwargs: str) -> Mapping[str, str]:
    """Створює read-only мапу метаданих з копії переданих значень."""
    return MappingProxyType(dict(kwargs))


@dataclass(frozen=True, slots=True)
class RoutePoint:
    """Точка маршруту посилки."""

    city: str
    hub: str
    hours: float


@dataclass(frozen=True, slots=True)
class Parcel:
    """Посилка. Маршрут - tuple, мітки - frozenset, meta - MappingProxyType.

    Поле meta виключено з хешу (hash=False), бо MappingProxyType не хешується.
    """

    id: int
    sender: str
    weight_kg: float
    status: Status = "created"
    route: tuple[RoutePoint, ...] = ()
    labels: frozenset[str] = frozenset()
    meta: Mapping[str, str] = field(default_factory=make_meta, hash=False)

    def total_hours(self) -> float:
        """Сумарний час доставки за маршрутом."""
        return sum(p.hours for p in self.route)

    def with_point(self, point: RoutePoint) -> "Parcel":
        """Повертає нову посилку з доданою точкою маршруту."""
        return replace(self, route=self.route + (point,))

    def with_label(self, label: str) -> "Parcel":
        """Повертає нову посилку з доданою міткою."""
        return replace(self, labels=self.labels | frozenset({label}))

    def with_status(self, status: Status) -> "Parcel":
        """Повертає нову посилку з новим статусом."""
        return replace(self, status=status)
```

Файл core.py

core.py рядки 1-60

```python
"""Pure-оновлення та бізнес-функції системи доставки (ЛР3, варіант 9)."""

from collections.abc import Iterable, Mapping
from dataclasses import replace
from functools import lru_cache
from types import MappingProxyType

from model import Parcel, RoutePoint, Status, make_meta

ALLOWED_TRANSITIONS: Mapping[Status, frozenset[Status]] = MappingProxyType(
    {
        "created": frozenset({"in_transit", "returned"}),
        "in_transit": frozenset({"delivered", "returned"}),
        "delivered": frozenset(),
        "returned": frozenset(),
    }
)


def add_route_point(parcel: Parcel, city: str, hub: str, hours: float) -> Parcel:
    """Pure update: додає точку маршруту (конкатенація tuple)."""
    return parcel.with_point(RoutePoint(city, hub, hours))


def add_label(parcel: Parcel, label: str) -> Parcel:
    """Pure update: додає мітку (операція frozenset)."""
    return parcel.with_label(label)


def remove_label(parcel: Parcel, label: str) -> Parcel:
    """Pure update: прибирає мітку (різниця frozenset)."""
    return replace(parcel, labels=parcel.labels - frozenset({label}))


def change_status(parcel: Parcel, status: Status) -> Parcel:
    """Pure update: змінює статус, якщо такий перехід дозволено."""
    if status not in ALLOWED_TRANSITIONS[parcel.status]:
        raise ValueError(f"Недозволений перехід: {parcel.status} -> {status}")
    return parcel.with_status(status)


def update_meta(parcel: Parcel, **changes: str) -> Parcel:
    """Pure update: нова read-only мапа метаданих з доданими значеннями."""
    return replace(parcel, meta=make_meta(**{**parcel.meta, **changes}))


@lru_cache(maxsize=None)
def delivery_cost(weight_kg: float, points: int, express: bool) -> float:
    """Вартість доставки: 40 грн + 15 грн/кг + 25 грн за точку, express x1.5."""
    base = 40.0 + 15.0 * weight_kg + 25.0 * points
    return round(base * (1.5 if express else 1.0), 2)


def parcel_cost(parcel: Parcel) -> float:
    """Вартість конкретної посилки (кешований розрахунок)."""
    return delivery_cost(
        parcel.weight_kg, len(parcel.route), "express" in parcel.labels
    )
```

core.py рядки 61-76

```python
def freeze(obj: object) -> object:
    """Глибоке заморожування: dict -> proxy, list -> tuple, set -> frozenset."""
    if isinstance(obj, Mapping):
        return MappingProxyType({k: freeze(v) for k, v in obj.items()})
    if isinstance(obj, (set, frozenset)):
        return frozenset(freeze(x) for x in obj)
    if isinstance(obj, (list, tuple)):
        return tuple(freeze(x) for x in obj)
    return obj


def index_by_id_and_labels(
    parcels: Iterable[Parcel],
) -> Mapping[tuple[int, frozenset[str]], Parcel]:
    """Індекс посилок за ключем (id, labels) - працює завдяки хешованості."""
    return MappingProxyType({(p.id, p.labels): p for p in parcels})
```

Файл main.py

main.py рядки 1-38

```python
"""Точка входу лабораторної роботи №3, варіант 9."""

from core import add_label, add_route_point, change_status, parcel_cost, update_meta
from model import Parcel, make_meta


def run() -> None:
    """Демонструє pure-оновлення незмінної посилки."""
    parcel = Parcel(
        id=101,
        sender="Нова пошта",
        weight_kg=2.5,
        meta=make_meta(source="web", region="Lviv"),
    )
    step1 = add_route_point(parcel, "Львів", "Сортувальний центр", 4.0)
    step2 = add_route_point(step1, "Київ", "Хаб-2", 6.5)
    step3 = add_label(step2, "express")
    step4 = change_status(step3, "in_transit")
    step5 = update_meta(step4, courier="Petro")

    for name, p in [("початкова", parcel), ("після оновлень", step5)]:
        print(f"--- {name} ---")
        print(f"статус: {p.status}, мітки: {sorted(p.labels)}")
        print(f"маршрут: {[pt.city for pt in p.route]}, годин: {p.total_hours()}")
        print(f"meta: {dict(p.meta)}, вартість: {parcel_cost(p)} грн")
    print(
        f"\noригінал не змінився: {parcel.route == () and parcel.status == 'created'}"
    )
    print(f"нова копія: {step5 is not parcel}, hash: {hash(step5) != hash(parcel)}")


def main() -> None:
    """Запуск демонстрації незмінної моделі доставки."""
    run()


if __name__ == "__main__":
    main()
```

Файл tests/test_immutability.py

test_immutability.py рядки 1-62

```python
"""Тести незмінності моделі доставки (ЛР3, варіант 9)."""

from dataclasses import FrozenInstanceError, replace
from types import MappingProxyType

import pytest

from core import (
    add_label,
    add_route_point,
    change_status,
    delivery_cost,
    freeze,
    index_by_id_and_labels,
    parcel_cost,
    remove_label,
    update_meta,
)
from model import Parcel, RoutePoint, make_meta


@pytest.fixture
def parcel() -> Parcel:
    """Базова посилка для тестів."""
    return Parcel(
        id=1,
        sender="Shop",
        weight_kg=2.0,
        route=(RoutePoint("Lviv", "Hub-1", 3.0),),
        labels=frozenset({"fragile"}),
        meta=make_meta(source="web"),
    )


def test_no_mutation_when_adding_point(parcel: Parcel) -> None:
    """Тест: додавання точки маршруту не змінює оригінал."""
    original = parcel
    snapshot = replace(parcel)
    new = add_route_point(parcel, "Kyiv", "Hub-2", 5.0)
    assert parcel is original
    assert parcel == snapshot
    assert new is not parcel
    assert len(new.route) == 2 and len(parcel.route) == 1
    assert new.total_hours() == parcel.total_hours() + 5.0


def test_labels_pure_update(parcel: Parcel) -> None:
    """Тест: мітки додаються і прибираються без мутації."""
    tagged = add_label(parcel, "express")
    assert tagged.labels == frozenset({"fragile", "express"})
    assert parcel.labels == frozenset({"fragile"})
    assert remove_label(tagged, "fragile").labels == frozenset({"express"})


def test_status_transitions(parcel: Parcel) -> None:
    """Тест: зміна статусу і заборонені переходи."""
    moving = change_status(parcel, "in_transit")
    assert moving.status == "in_transit" and parcel.status == "created"
    assert change_status(moving, "delivered").status == "delivered"
    with pytest.raises(ValueError):
        change_status(parcel, "delivered")
```

test_immutability.py рядки 63-122

```python
def test_frozen_fields(parcel: Parcel) -> None:
    """Тест: поля frozen dataclass змінити не можна."""
    with pytest.raises(FrozenInstanceError):
        parcel.status = "returned"  # type: ignore[misc]


def test_meta_is_read_only(parcel: Parcel) -> None:
    """Тест: meta - read-only MappingProxyType."""
    assert isinstance(parcel.meta, MappingProxyType)
    with pytest.raises(TypeError):
        parcel.meta["source"] = "changed"  # type: ignore[index]


def test_make_meta_copies_source() -> None:
    """Тест: make_meta копіює джерело, тож зміна dict не впливає на proxy."""
    source = {"source": "web"}
    meta = make_meta(**source)
    source["source"] = "changed"
    assert meta["source"] == "web"


def test_update_meta_returns_new_copy(parcel: Parcel) -> None:
    """Тест: update_meta повертає нову посилку з новою мапою."""
    updated = update_meta(parcel, courier="Ivan")
    assert dict(updated.meta) == {"source": "web", "courier": "Ivan"}
    assert dict(parcel.meta) == {"source": "web"}


def test_hashability(parcel: Parcel) -> None:
    """Тест: Parcel і RoutePoint хешуються, працюють як ключі set/dict."""
    parcels = {parcel, add_label(parcel, "express")}
    assert len(parcels) == 2
    assert hash(RoutePoint("Lviv", "Hub-1", 3.0)) == hash(parcel.route[0])
    index = index_by_id_and_labels(parcels)
    assert index[(1, frozenset({"fragile"}))] == parcel


def test_mapping_proxy_is_not_hashable() -> None:
    """Тест: саме MappingProxyType заважає хешуванню, тому meta виключено з hash."""
    with pytest.raises(TypeError):
        hash(make_meta(a="b"))


def test_freeze_deep() -> None:
    """Тест: глибоке заморожування вкладених структур."""
    frozen = freeze({"tags": {"a"}, "points": [1, [2, 3]], "info": {"k": "v"}})
    assert isinstance(frozen, MappingProxyType)
    assert frozen["tags"] == frozenset({"a"})
    assert frozen["points"] == (1, (2, 3))
    assert isinstance(frozen["info"], MappingProxyType)


def test_cost_cached(parcel: Parcel) -> None:
    """Тест: вартість доставки рахується правильно і кешується lru_cache."""
    delivery_cost.cache_clear()
    assert parcel_cost(parcel) == 95.0
    assert parcel_cost(add_label(parcel, "express")) == 142.5
    parcel_cost(parcel)
    assert delivery_cost.cache_info().hits == 1
```

Файл tests/test_performance.py

test_performance.py рядки 1-54

```python
"""Порівняння часу: мутабельна версія (до) і незмінна (після).

Запуск: python -m pytest --durations=5
"""

from typing import Any

from core import add_label, add_route_point
from model import Parcel

ROUTE_STEPS = 2_000
LABEL_UPDATES = 20_000


def add_point_mutable(parcel: dict[str, Any], city: str, hours: float) -> None:
    """Мутабельна версія «до»: дописує точку маршруту на місці."""
    parcel["route"].append({"city": city, "hub": "Hub", "hours": hours})


def add_label_mutable(parcel: dict[str, Any], label: str) -> None:
    """Мутабельна версія «до»: додає мітку на місці."""
    parcel["labels"].add(label)


def test_before_route_mutable() -> None:
    """До: ROUTE_STEPS in-place додавань точок маршруту."""
    parcel: dict[str, Any] = {"id": 1, "route": [], "labels": set()}
    for i in range(ROUTE_STEPS):
        add_point_mutable(parcel, f"C{i}", 1.0)
    assert len(parcel["route"]) == ROUTE_STEPS


def test_after_route_immutable() -> None:
    """Після: ROUTE_STEPS pure-оновлень, кожне створює новий tuple маршруту."""
    parcel = Parcel(id=1, sender="Shop", weight_kg=1.0)
    for i in range(ROUTE_STEPS):
        parcel = add_route_point(parcel, f"C{i}", "Hub", 1.0)
    assert len(parcel.route) == ROUTE_STEPS


def test_before_labels_mutable() -> None:
    """До: LABEL_UPDATES in-place додавань міток."""
    parcel: dict[str, Any] = {"id": 1, "route": [], "labels": set()}
    for i in range(LABEL_UPDATES):
        add_label_mutable(parcel, f"L{i % 10}")
    assert len(parcel["labels"]) == 10


def test_after_labels_immutable() -> None:
    """Після: LABEL_UPDATES pure-оновлень міток через frozenset."""
    parcel = Parcel(id=1, sender="Shop", weight_kg=1.0)
    for i in range(LABEL_UPDATES):
        parcel = add_label(parcel, f"L{i % 10}")
    assert len(parcel.labels) == 10
```

Запуск програми

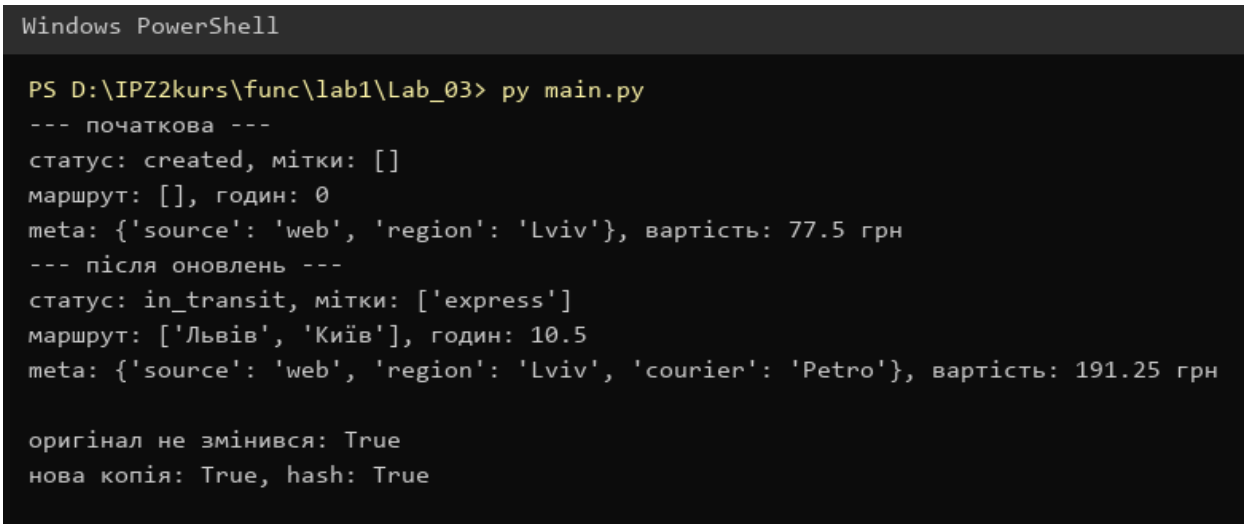


Рис. 1. Вивід програми (python main.py)

Виконання тестів

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_03> py -m pytest -v --durations=5
============================= test session starts =============================
platform win32 -- Python 3.13.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\LENOVO\AppData\Local\Programs\Python\Python313\python.exe
rootdir: D:\IPZ2kurs\func\lab1\Lab_03
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.12.4
collecting ... collected 15 items

tests/test_immutability.py::test_no_mutation_when_adding_point PASSED    [  6%]
tests/test_immutability.py::test_labels_pure_update PASSED               [ 13%]
tests/test_immutability.py::test_status_transitions PASSED               [ 20%]
tests/test_immutability.py::test_frozen_fields PASSED                    [ 26%]
tests/test_immutability.py::test_meta_is_read_only PASSED                [ 33%]
tests/test_immutability.py::test_make_meta_copies_source PASSED          [ 40%]
tests/test_immutability.py::test_update_meta_returns_new_copy PASSED     [ 46%]
tests/test_immutability.py::test_hashability PASSED                      [ 53%]
tests/test_immutability.py::test_mapping_proxy_is_not_hashable PASSED    [ 60%]
tests/test_immutability.py::test_freeze_deep PASSED                      [ 66%]
tests/test_immutability.py::test_cost_cached PASSED                      [ 73%]
tests/test_performance.py::test_before_route_mutable PASSED              [ 80%]
tests/test_performance.py::test_after_route_immutable PASSED             [ 86%]
tests/test_performance.py::test_before_labels_mutable PASSED             [ 93%]
tests/test_performance.py::test_after_labels_immutable PASSED            [100%]

============================== slowest 5 durations ==============================
0.08s call     tests/test_performance.py::test_after_labels_immutable
0.02s call     tests/test_performance.py::test_after_route_immutable
0.01s call     tests/test_performance.py::test_before_labels_mutable

(2 durations < 0.005s hidden.  Use -vv to show these durations.)
============================== 15 passed in 0.17s ==============================
```

Рис. 2. Результат запуску pytest

Перевірка типів і стилю

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_03> py -m mypy .
Success: no issues found in 5 source files
PS D:\IPZ2kurs\func\lab1\Lab_03> py -m ruff check .
All checks passed!
PS D:\IPZ2kurs\func\lab1\Lab_03> py -m black --check .
All done!
5 files would be left unchanged.
```

Рис. 3. mypy, ruff та black без зауважень

Висновок:

Під час виконання роботи мутабельну модель посилки перетворено на незмінну за допомогою frozen dataclass, tuple, frozenset і MappingProxyType. Усі зміни стану реалізовано як pure updates, тому оригінальні об'єкти ніколи не змінюються, а посилки можна використовувати як ключі set і dict. Вимірювання показали, що ціна незмінності - створення копій: на тисячах оновлень це мілісекунди, тож для практичних обсягів вона несуттєва порівняно з виграшем у надійності.